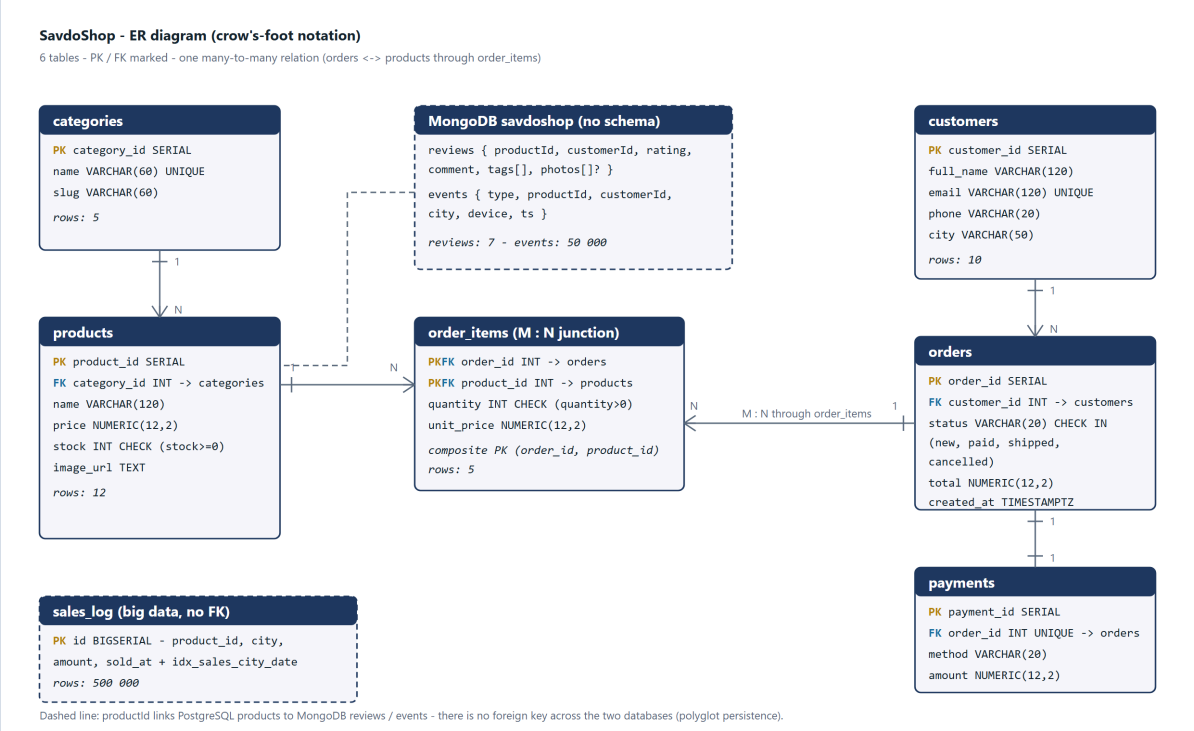
# SavdoShop - Week 1 results

Big Data course - Deliverable 1 (PostgreSQL + MongoDB)
Every output below was produced by running the scripts in this folder on one PC, so the numbers are real.
Generated: 05/11/2026, 03:14:28

## 0. Row counts after running the scripts

> 01_schema.sql + 02_seed.sql give 5 categories / 10 customers / 12 products / 3 orders / 5 order_items / 2 payments. The counts below are taken after 03_queries.sql ran, so Q1-Q4 (one customer added and deleted), Q3 (Shoes -10%) and Q13 (the new order 4) are already visible: cats = 5 · custs = 10 · prods = 12 · ords = 4 · lines = 6 · pays = 2 · sales_log = 500 000 · MongoDB reviews = 7, events = 50 000

## 1. ER diagram




## 2. Twenty SQL queries (03_queries.sql)

### Q1 Business question: register a new customer so we can serve them?

Type: CRUD - CREATE

```
INSERT INTO customers (full_name, email, city)
VALUES ('Student Test', 'student@example.uz', 'Tashkent')
RETURNING customer_id;
```

| customer_id |
|---|
| 11 |

28 ms on this PC

### Q2 Business question: which products are affordable (under 400 000 so'm)?

Type: CRUD - READ

```
SELECT product_id, name, price
FROM products
WHERE price < 400000
ORDER BY price ASC;
```

| product_id | name | price |
|---|---|---|
| 7 | Argan hair oil | 95000.00 |
| 8 | Rose face cream | 120000.00 |
| 2 | Cotton shirt | 180000.00 |
| 12 | Power bank 20000 mAh | 280000.00 |
| 9 | Silver earrings | 310000.00 |
| 11 | Wireless earbuds | 350000.00 |
| 5 | Running shoes | 380000.00 |

4 ms on this PC

### Q3 Business question: give Shoes a 10% discount before a sale week?

Type: CRUD - UPDATE

```
UPDATE products
SET price = price * 0.9
WHERE category_id = (SELECT category_id FROM categories WHERE name = 'Shoes');
```

(UPDATE, 3 row(s) - no rows returned)

4 ms on this PC

### Q4 Business question: remove the test row we no longer need?

Type: CRUD - DELETE

```
DELETE FROM customers WHERE email = 'student@example.uz';
```

(DELETE, 1 row(s) - no rows returned)

13 ms on this PC

### Q5 Business question: show the catalogue with a readable category name?

Type: INNER JOIN (2 tables)

```
SELECT p.name AS product, c.name AS category, p.price
FROM products p
JOIN categories c ON c.category_id = p.category_id
ORDER BY c.name, p.name;
```

| product | category | price |
|---|---|---|
| Argan hair oil | Beauty | 95000.00 |
| Rose face cream | Beauty | 120000.00 |
| Atlas silk dress | Clothing | 650000.00 |
| Cotton shirt | Clothing | 180000.00 |
| Winter chapan | Clothing | 950000.00 |
| Power bank 20000 mAh | Electronics | 280000.00 |
| Wireless earbuds | Electronics | 350000.00 |
| Gold ring 585 | Jewelry | 2400000.00 |
| Silver earrings | Jewelry | 310000.00 |
| Classic boots | Shoes | 504000.00 |
| Leather sneakers | Shoes | 378000.00 |
| Running shoes | Shoes | 342000.00 |

2 ms on this PC

### Q6 Business question: who bought what - the full order detail line?

Type: INNER JOIN (4 tables)

```
SELECT o.order_id, cu.full_name, p.name AS product, oi.quantity,
       oi.unit_price * oi.quantity AS line_total
FROM orders o
JOIN customers cu ON cu.customer_id = o.customer_id
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
ORDER BY o.order_id;
```

| order_id | full_name | product | quantity | line_total |
|---|---|---|---|---|
| 1 | Aziza Karimova | Atlas silk dress | 1 | 650000.00 |
| 1 | Aziza Karimova | Cotton shirt | 1 | 180000.00 |
| 2 | Bekzod Tursunov | Argan hair oil | 1 | 95000.00 |
| 2 | Bekzod Tursunov | Leather sneakers | 1 | 420000.00 |
| 3 | Jasur Aliyev | Gold ring 585 | 1 | 2400000.00 |

11 ms on this PC

### Q7 Business question: which customers have NEVER ordered (to target them)?

Type: LEFT JOIN + IS NULL

```
SELECT cu.full_name, cu.city
FROM customers cu
LEFT JOIN orders o ON o.customer_id = cu.customer_id
WHERE o.order_id IS NULL;
```

| full_name | city |
|---|---|
| Sardor Qodirov | Samarkand |
| Malika Yusupova | Namangan |
| Timur Saidov | Tashkent |
| Otabek Ergashev | Andijan |
| Dilnoza Rahimova | Bukhara |
| Zarina Islomova | Fergana |
| Sevara Nazarova | Khiva |

3 ms on this PC

### Q8 Business question: how wide is the range and price level per category?

Type: LEFT JOIN + GROUP BY

```
SELECT c.name, COUNT(p.product_id) AS products, ROUND(AVG(p.price)) AS avg_price
FROM categories c
LEFT JOIN products p ON p.category_id = c.category_id
GROUP BY c.name
ORDER BY products DESC;
```

| name | products | avg_price |
|---|---|---|
| Clothing | 3 | 593333 |
| Shoes | 3 | 408000 |
| Electronics | 2 | 315000 |
| Beauty | 2 | 107500 |
| Jewelry | 2 | 1355000 |

3 ms on this PC

### Q9 Business question: which cities bring real money (over 500 000 so'm)?

Type: GROUP BY + HAVING

```
SELECT cu.city, SUM(o.total) AS revenue, COUNT(*) AS orders
FROM orders o
JOIN customers cu ON cu.customer_id = o.customer_id
WHERE o.status <> 'cancelled'
GROUP BY cu.city
HAVING SUM(o.total) > 500000
ORDER BY revenue DESC;
```

| city | revenue | orders |
|---|---|---|
| Tashkent | 3230000.00 | 2 |
| Samarkand | 515000.00 | 1 |

3 ms on this PC

### Q10 Business question: what are our top 5 best-selling products?

Type: GROUP BY + ORDER BY + LIMIT

```
SELECT p.name, SUM(oi.quantity) AS sold
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.name
ORDER BY sold DESC
LIMIT 5;
```

| name | sold |
|---|---|
| Leather sneakers | 1 |
| Atlas silk dress | 1 |
| Cotton shirt | 1 |
| Gold ring 585 | 1 |
| Argan hair oil | 1 |

5 ms on this PC

### Q11 Business question: give the website one reusable catalogue query?

Type: VIEW (CREATE OR REPLACE)

```
CREATE OR REPLACE VIEW v_product_catalog AS
SELECT p.product_id, p.name, p.price, p.stock, p.image_url, c.name AS category
FROM products p
JOIN categories c ON c.category_id = p.category_id;
```

(CREATE - no rows returned)

5 ms on this PC

### Q11 also use it: the Jewelry shelf of the website

```
SELECT * FROM v_product_catalog WHERE category = 'Jewelry';
```

| product_id | name | price | stock | image_url | category |
|---|---|---|---|---|---|
| 9 | Silver earrings | 310000.00 | 20 | /placeholder.svg?text=Earrings | Jewelry |
| 10 | Gold ring 585 | 2400000.00 | 5 | /placeholder.svg?text=Gold+ring | Jewelry |

2 ms on this PC

### Q12 Business question: what is the price ranking inside each category?

Type: WINDOW FUNCTION (RANK OVER PARTITION BY)

```
SELECT category, name, price,
       RANK() OVER (PARTITION BY category ORDER BY price DESC) AS price_rank
FROM v_product_catalog;
```

| category | name | price | price_rank |
|---|---|---|---|
| Beauty | Rose face cream | 120000.00 | 1 |
| Beauty | Argan hair oil | 95000.00 | 2 |
| Clothing | Winter chapan | 950000.00 | 1 |
| Clothing | Atlas silk dress | 650000.00 | 2 |
| Clothing | Cotton shirt | 180000.00 | 3 |
| Electronics | Wireless earbuds | 350000.00 | 1 |
| Electronics | Power bank 20000 mAh | 280000.00 | 2 |
| Jewelry | Gold ring 585 | 2400000.00 | 1 |
| Jewelry | Silver earrings | 310000.00 | 2 |
| Shoes | Classic boots | 504000.00 | 1 |
| Shoes | Leather sneakers | 378000.00 | 2 |
| Shoes | Running shoes | 342000.00 | 3 |

3 ms on this PC

### Q13 Business question: place an order without ever breaking the stock count?

Type: TRANSACTION (BEGIN...COMMIT)

```
BEGIN;
   INSERT INTO orders (customer_id) VALUES (3) RETURNING order_id;
   INSERT INTO order_items (order_id, product_id, quantity, unit_price)
   SELECT currval('orders_order_id_seq'), product_id, 1, price
   FROM products WHERE product_id = 4;
   UPDATE products SET stock = stock - 1 WHERE product_id = 4;
   UPDATE orders SET total = (SELECT SUM(quantity * unit_price) FROM order_items
                              WHERE order_id = currval('orders_order_id_seq'))
   WHERE order_id = currval('orders_order_id_seq');
COMMIT; -- or ROLLBACK; to cancel everything
```

(BEGIN/INSERT/INSERT/UPDATE/UPDATE/COMMIT, 4 row(s) - no rows returned)

10 ms on this PC

### Q13 check Business question: did the new order 4 store the discounted price?

Type: JOIN + scalar sub-query

```
SELECT o.order_id, o.customer_id, o.total, oi.product_id, oi.unit_price,
       (SELECT stock FROM products WHERE product_id = 4) AS stock_now
FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
WHERE o.order_id = 4;
```

| order_id | customer_id | total | product_id | unit_price | stock_now |
|---|---|---|---|---|---|
| 4 | 3 | 378000.00 | 4 | 378000.00 | 24 |

2 ms on this PC

### Q14 Business question: which orders are already paid and how much came in?

Type: INNER JOIN (3 tables)

```
SELECT o.order_id, cu.full_name, o.status, p.method, p.amount, p.paid_at
FROM payments p
JOIN orders o ON o.order_id = p.order_id
JOIN customers cu ON cu.customer_id = o.customer_id
ORDER BY p.paid_at;
```

| order_id | full_name | status | method | amount | paid_at |
|---|---|---|---|---|---|
| 1 | Aziza Karimova | paid | payme | 830000.00 | 2026-11-04T22:02:56.730Z |
| 2 | Bekzod Tursunov | shipped | card | 515000.00 | 2026-11-04T22:02:56.730Z |

12 ms on this PC

### Q15 Business question: how many orders are in each status, and their average?

Type: GROUP BY (per status)

```
SELECT status,
       COUNT(*) AS orders,
       ROUND(AVG(total)) AS avg_total,
       MIN(created_at) AS first_order,
       MAX(created_at) AS last_order
FROM orders
GROUP BY status
ORDER BY orders DESC;
```

| status | orders | avg_total | first_order | last_order |
|---|---|---|---|---|
| new | 2 | 1389000 | 2026-11-04T22:02:56.730Z | 2026-11-04T22:02:57.204Z |
| paid | 1 | 830000 | 2026-11-04T22:02:56.730Z | 2026-11-04T22:02:56.730Z |
| shipped | 1 | 515000 | 2026-11-04T22:02:56.730Z | 2026-11-04T22:02:56.730Z |

3 ms on this PC

### Q16 Business question: which products cost more than the shop average price?

Type: SUB-QUERY (scalar sub-select in WHERE)

```
SELECT name, price,
       (SELECT ROUND(AVG(price)) FROM products) AS shop_avg,
       price - (SELECT ROUND(AVG(price)) FROM products) AS above_avg
FROM products
WHERE price > (SELECT AVG(price) FROM products)
ORDER BY price DESC;
```

| name | price | shop_avg | above_avg |
|---|---|---|---|
| Gold ring 585 | 2400000.00 | 546583 | 1853417.00 |
| Winter chapan | 950000.00 | 546583 | 403417.00 |
| Atlas silk dress | 650000.00 | 546583 | 103417.00 |

2 ms on this PC

### Q17 Business question: how does each customer's spending grow order by order?

Type: WINDOW FUNCTION (running SUM OVER PARTITION BY ... ORDER BY)

```
SELECT o.customer_id, cu.full_name, o.order_id, o.created_at, o.total,
       SUM(o.total) OVER (PARTITION BY o.customer_id ORDER BY o.created_at) AS running_total
FROM orders o
JOIN customers cu ON cu.customer_id = o.customer_id
ORDER BY o.customer_id, o.created_at;
```

| customer_id | full_name | order_id | created_at | total | running_total |
|---|---|---|---|---|---|
| 1 | Aziza Karimova | 1 | 2026-11-04T22:02:56.730Z | 830000.00 | 830000.00 |
| 2 | Bekzod Tursunov | 2 | 2026-11-04T22:02:56.730Z | 515000.00 | 515000.00 |
| 3 | Dilnoza Rahimova | 4 | 2026-11-04T22:02:57.204Z | 378000.00 | 378000.00 |
| 4 | Jasur Aliyev | 3 | 2026-11-04T22:02:56.730Z | 2400000.00 | 2400000.00 |

2 ms on this PC

### Q18 Business question: which weekday brings the most revenue (staffing plan)?

Type: FREE - date_trunc + TO_CHAR + GROUP BY over the 500 000-row log

```
SELECT EXTRACT(ISODOW FROM sold_at)::int AS weekday_number,
       TO_CHAR(sold_at, 'Day') AS weekday,
       COUNT(*) AS sales,
       ROUND(SUM(amount)) AS revenue
FROM sales_log
GROUP BY 1, 2
ORDER BY revenue DESC;
```

| weekday_number | weekday | sales | revenue |
|---|---|---|---|
| 2 | Tuesday | 72527 | 76227854000 |
| 1 | Monday | 71761 | 75273517000 |
| 3 | Wednesday | 71442 | 74950431000 |
| 7 | Sunday | 71290 | 74928696000 |
| 6 | Saturday | 71251 | 74881569000 |
| 5 | Friday | 71013 | 74599121000 |
| 4 | Thursday | 70716 | 74263007000 |

5390 ms on this PC

### Q19 Business question: which customers spend more than the average customer?

Type: FREE - CTE (WITH) + JOIN

```
WITH spend AS (
    SELECT customer_id, SUM(total) AS total_spent
    FROM orders
    WHERE status <> 'cancelled'
    GROUP BY customer_id
)
SELECT cu.full_name, cu.city, s.total_spent
FROM spend s
JOIN customers cu ON cu.customer_id = s.customer_id
WHERE s.total_spent > (SELECT AVG(total_spent) FROM spend)
ORDER BY s.total_spent DESC;
```

| full_name | city | total_spent |
|---|---|---|
| Jasur Aliyev | Tashkent | 2400000.00 |

6 ms on this PC

### Q20 Business question: how many different customers tried each product line?

Type: FREE - GROUP BY + COUNT(DISTINCT) + CASE segment

```
SELECT c.name AS category,
       COUNT(DISTINCT oi.product_id) AS products_sold,
       COUNT(DISTINCT o.customer_id) AS unique_customers,
       SUM(oi.quantity) AS units,
       CASE WHEN SUM(oi.quantity) >= 3 THEN 'hit' ELSE 'normal' END AS demand
FROM order_items oi
JOIN products p ON p.product_id = oi.product_id
JOIN categories c ON c.category_id = p.category_id
JOIN orders o ON o.order_id = oi.order_id
GROUP BY c.name
ORDER BY units DESC;
```

| category | products_sold | unique_customers | units | demand |
|---|---|---|---|---|
| Clothing | 2 | 1 | 2 | normal |
| Shoes | 1 | 2 | 2 | normal |
| Beauty | 1 | 1 | 1 | normal |
| Jewelry | 1 | 1 | 1 | normal |

3 ms on this PC

## 3. Big data and indexes (04_big_data.sql)

### Step 0: create an empty table for the sales log

```
DROP TABLE IF EXISTS sales_log CASCADE;

CREATE TABLE sales_log (
    id         BIGSERIAL PRIMARY KEY,
    product_id INT NOT NULL,
    city       VARCHAR(50) NOT NULL,
    amount     NUMERIC(12,2) NOT NULL,
    sold_at    TIMESTAMPTZ NOT NULL
);
```

(DROP/CREATE - no rows returned)

237 ms on this PC

### Step 1: generate 500 000 random sales in one statement

```
INSERT INTO sales_log (product_id, city, amount, sold_at)
SELECT 1 + floor(random() * 12)::int,
       (ARRAY['Tashkent','Samarkand','Bukhara','Namangan','Andijan',
              'Fergana','Khiva','Nukus'])[1 + floor(random() * 8)::int],
       round((50000 + random() * 2000000)::numeric, -3),
       now() - random() * interval '365 days'
FROM generate_series(1, 500000);

ANALYZE sales_log;                        -- refresh planner statistics

SELECT COUNT(*) FROM sales_log;
```

| count |
|---|
| 500000 |

34110 ms on this PC

### Step 2: measure a query WITHOUT an index (look for "Seq Scan" + Execution Time)

```
EXPLAIN ANALYZE
SELECT COUNT(*), SUM(amount)
FROM sales_log
WHERE city = 'Khiva' AND sold_at > now() - interval '30 days';
```

```
Finalize Aggregate  (cost=9344.74..9344.75 rows=1 width=40) (actual time=1026.559..1109.426
rows=1.00 loops=1)
  Buffers: shared hit=4167
  ->  Gather  (cost=9344.51..9344.72 rows=2 width=40) (actual time=138.398..1109.387 rows=3.00
loops=1)
        Workers Planned: 2
        Workers Launched: 2
        Buffers: shared hit=4167
        ->  Partial Aggregate  (cost=8344.51..8344.52 rows=1 width=40) (actual time=45.931..45.931
rows=1.00 loops=3)
              Buffers: shared hit=4167
              ->  Parallel Seq Scan on sales_log  (cost=0.00..8333.67 rows=2168 width=6) (actual
time=0.015..44.857 rows=1696.33 loops=3)
                    Filter: (((city)::text = 'Khiva'::text) AND (sold_at > (now() - '30
days'::interval)))
                    Rows Removed by Filter: 164970
                    Buffers: shared hit=4167
Planning:
  Buffers: shared hit=49
Planning Time: 3.084 ms
Execution Time: 1110.070 ms
```

1120 ms on this PC

### Step 3: add a composite index and measure again (look for "Index" / "Bitmap")

```
CREATE INDEX idx_sales_city_date ON sales_log (city, sold_at);

EXPLAIN ANALYZE
SELECT COUNT(*), SUM(amount)
FROM sales_log
WHERE city = 'Khiva' AND sold_at > now() - interval '30 days';
-- Write both execution times in your report and explain the difference.
```

```
Aggregate  (cost=4663.38..4663.39 rows=1 width=40) (actual time=82.188..82.190 rows=1.00 loops=1)
  Buffers: shared hit=2899 read=22
  ->  Bitmap Heap Scan on sales_log  (cost=145.76..4637.36 rows=5203 width=6) (actual
time=35.851..78.411 rows=5089.00 loops=1)
        Recheck Cond: (((city)::text = 'Khiva'::text) AND (sold_at > (now() - '30
days'::interval)))
        Heap Blocks: exact=2899
        Buffers: shared hit=2899 read=22
        ->  Bitmap Index Scan on idx_sales_city_date  (cost=0.00..144.46 rows=5203 width=0) (actual
time=29.428..29.428 rows=5089.00 loops=1)
              Index Cond: (((city)::text = 'Khiva'::text) AND (sold_at > (now() - '30
days'::interval)))
              Index Searches: 1
              Buffers: shared read=22
Planning:
  Buffers: shared hit=4 read=1
Planning Time: 13.353 ms
Execution Time: 82.372 ms
```

9377 ms on this PC

### Step 4: monthly revenue trend (time-series analytics)

```
SELECT date_trunc('month', sold_at)::date AS month,
       COUNT(*)                           AS sales,
       ROUND(SUM(amount))                 AS revenue
FROM sales_log
GROUP BY 1
ORDER BY 1;
```

| month | sales | revenue |
|---|---|---|
| 2025-10-31T19:00:00.000Z | 35689 | 37472342000 |
| 2025-11-30T19:00:00.000Z | 42759 | 44968619000 |
| 2025-12-31T19:00:00.000Z | 42361 | 44448124000 |
| 2026-01-31T19:00:00.000Z | 38146 | 39982377000 |
| 2026-02-28T19:00:00.000Z | 42556 | 44714233000 |
| 2026-03-31T19:00:00.000Z | 40879 | 42862628000 |
| 2026-04-30T19:00:00.000Z | 42500 | 44614714000 |
| 2026-05-31T19:00:00.000Z | 41217 | 43148756000 |
| 2026-06-30T19:00:00.000Z | 42303 | 44476702000 |
| 2026-07-31T19:00:00.000Z | 42282 | 44399165000 |
| 2026-08-31T19:00:00.000Z | 41303 | 43448309000 |
| 2026-09-30T19:00:00.000Z | 42255 | 44478644000 |
| 2026-10-31T19:00:00.000Z | 5750 | 6091974000 |

1096 ms on this PC

### Step 5: materialized view = pre-computed summary for fast dashboards

```
DROP MATERIALIZED VIEW IF EXISTS mv_city_daily;
CREATE MATERIALIZED VIEW mv_city_daily AS
SELECT city, sold_at::date AS day, COUNT(*) AS sales, SUM(amount) AS revenue
FROM sales_log
GROUP BY city, sold_at::date;

SELECT * FROM mv_city_daily WHERE city = 'Tashkent' ORDER BY day DESC LIMIT 7;

REFRESH MATERIALIZED VIEW mv_city_daily; -- run again after new data arrives
```

(DROP/SELECT/SELECT/REFRESH - no rows returned)

2292 ms on this PC

### Step 6: top 3 products in every city (window function over 500k rows)

```
SELECT * FROM (
      SELECT city, product_id, SUM(amount) AS revenue,
             ROW_NUMBER() OVER (PARTITION BY city ORDER BY SUM(amount) DESC) AS rn
      FROM sales_log
      GROUP BY city, product_id
) t
WHERE rn <= 3
ORDER BY city, rn;
```

| city | product_id | revenue | rn |
|---|---|---|---|
| Andijan | 10 | 5587649000.00 | 1 |
| Andijan | 4 | 5568704000.00 | 2 |
| Andijan | 3 | 5520744000.00 | 3 |
| Bukhara | 9 | 5629087000.00 | 1 |
| Bukhara | 1 | 5611279000.00 | 2 |
| Bukhara | 8 | 5590312000.00 | 3 |
| Fergana | 9 | 5648616000.00 | 1 |
| Fergana | 3 | 5617641000.00 | 2 |
| Fergana | 5 | 5566800000.00 | 3 |
| Khiva | 2 | 5611707000.00 | 1 |
| Khiva | 5 | 5533756000.00 | 2 |
| Khiva | 1 | 5495030000.00 | 3 |
| Namangan | 5 | 5651747000.00 | 1 |
| Namangan | 7 | 5586791000.00 | 2 |
| Namangan | 1 | 5553820000.00 | 3 |
| Nukus | 1 | 5578792000.00 | 1 |
| Nukus | 7 | 5538735000.00 | 2 |
| Nukus | 6 | 5495693000.00 | 3 |
| Samarkand | 10 | 5579532000.00 | 1 |
| Samarkand | 6 | 5541969000.00 | 2 |
| Samarkand | 5 | 5511975000.00 | 3 |
| Tashkent | 7 | 5689678000.00 | 1 |
| Tashkent | 3 | 5513164000.00 | 2 |
| Tashkent | 9 | 5491592000.00 | 3 |

837 ms on this PC

## 4. MongoDB queries (mongo_queries.js, M1-M14)

### counts

```
{
  reviews: 7,
  events: 50000
}
```

### M1

```
{
  acknowledged: true,
  insertedId: ObjectId('6aebac03c0cf34dba073a84e')
}
```

### M2

```
[
  {
    productId: 2,
    comment: 'Great shirt'
  },
  {
    productId: 1,
    comment: 'Beautiful atlas, fast delivery'
  },
  {
    productId: 4,
    comment: 'Very comfortable'
  },
  {
    productId: 10,
    comment: 'Perfect gift'
  }
]
```

### M3

```
[
  {
    productId: 1,
    rating: 4,
    tags: [
      'size'
    ]
  },
  {
    productId: 4,
    rating: 3,
    tags: [
      'size'
    ]
  },
  {
    productId: 11,
    rating: 2,
    tags: [
      'battery'
    ]
  }
]
```

### M4

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

### M4 after

```
{
  productId: 2,
  customerId: 5,
  rating: 5,
  comment: 'Great shirt',
  tags: [
    'quality',
    'recommended'
  ],
  createdAt: ISODate('2026-11-04T22:04:51.353Z'),
  helpful: 1
}
```

### M5

```
{
  acknowledged: true,
  deletedCount: 1
}
```

### M6

```
[
  {
    _id: 10,
    avgRating: 5,
    reviews: 1
  },
  {
    _id: 1,
    avgRating: 4.5,
    reviews: 2
  },
  {
    _id: 4,
    avgRating: 4,
    reviews: 2
  },
  {
    _id: 7,
    avgRating: 4,
    reviews: 1
  },
  {
    _id: 11,
    avgRating: 2,
    reviews: 1
  }
]
```

### M7

```
[
  {
    _id: 'view',
    count: 28753
  },
  {
    _id: 'add_to_cart',
    count: 14103
  },
  {
    _id: 'purchase',
    count: 7144
  }
]
```

### M8

```
[
  {
    _id: 12,
    views: 838
  },
  {
    _id: 11,
    views: 828
  },
  {
    _id: 8,
    views: 821
  },
  {
    _id: 5,
    views: 816
  },
  {
    _id: 1,
    views: 810
  }
]
```

**M9**

```
[
  {
    _id: {
      city: 'Andijan',
      device: 'ios'
    },
    n: 305
  },
  {
    _id: {
      city: 'Andijan',
      device: 'android'
    },
    n: 302
  },
  {
    _id: {
      city: 'Andijan',
      device: 'desktop'
    },
    n: 302
  },
  {
    _id: {
      city: 'Bukhara',
      device: 'android'
    },
    n: 307
  },
  {
    _id: {
      city: 'Bukhara',
      device: 'desktop'
    },
    n: 296
  },
  {
    _id: {
      city: 'Bukhara',
      device: 'ios'
    },
    n: 286
  },
  {
    _id: {
      city: 'Fergana',
      device: 'ios'
    },
    n: 331
  },
  {
    _id: {
      city: 'Fergana',
      device: 'android'
    },
    n: 303
  },
  {
    _id: {
      city: 'Fergana',
      device: 'desktop'
    },
    n: 257
  },
  {
    _id: {
      city: 'Khiva',
      device: 'ios'
    },
    n: 309
  },
  {
    _id: {
      city: 'Khiva',
      device: 'desktop'
    },
```

```
  n: 299
},
{
  _id: {
    city: 'Khiva',
    device: 'android'
  },
  n: 279
},
{
  _id: {
    city: 'Namangan',
    device: 'android'
  },
  n: 313
},
{
  _id: {
    city: 'Namangan',
    device: 'ios'
  },
  n: 306
},
{
  _id: {
    city: 'Namangan',
    device: 'desktop'
  },
  n: 286
},
{
  _id: {
    city: 'Nukus',
    device: 'ios'
  },
  n: 301
},
{
  _id: {
    city: 'Nukus',
    device: 'android'
  },
  n: 292
},
{
  _id: {
    city: 'Nukus',
    device: 'desktop'
  },
  n: 267
},
{
  _id: {
    city: 'Samarkand',
    device: 'android'
  },
  n: 326
},
{
  _id: {
    city: 'Samarkand',
    device: 'ios'
  },
  n: 305
},
{
  _id: {
    city: 'Samarkand',
    device: 'desktop'
  },
  n: 281
},
{
  _id: {
    city: 'Tashkent',
    device: 'android'
  },
  n: 323
```

```
  },
  {
    _id: {
      city: 'Tashkent',
      device: 'ios'
    },
    n: 292
  },
  {
    _id: {
      city: 'Tashkent',
      device: 'desktop'
    },
    n: 276
  }
]
```

**M10**

```
[
  {
    _id: '2026-10-21',
    events: 47
  },
  {
    _id: '2026-10-22',
    events: 555
  },
  {
    _id: '2026-10-23',
    events: 583
  },
  {
    _id: '2026-10-24',
    events: 592
  },
  {
    _id: '2026-10-25',
    events: 531
  },
  {
    _id: '2026-10-26',
    events: 584
  },
  {
    _id: '2026-10-27',
    events: 535
  },
  {
    _id: '2026-10-28',
    events: 525
  },
  {
    _id: '2026-10-29',
    events: 561
  },
  {
    _id: '2026-10-30',
    events: 513
  },
  {
    _id: '2026-10-31',
    events: 549
  },
  {
    _id: '2026-11-01',
    events: 582
  },
  {
    _id: '2026-11-02',
    events: 540
  },
  {
    _id: '2026-11-03',
    events: 554
  },
  {
    _id: '2026-11-04',
    events: 535
  }
]
```

### M11

```
[
  {
    _id: 'quality',
    count: 3
  },
  {
    _id: 'size',
    count: 2
  },
  {
    _id: 'delivery',
    count: 1
  },
  {
    _id: 'comfort',
    count: 1
  },
  {
    _id: 'gift',
    count: 1
  },
  {
    _id: 'battery',
    count: 1
  }
]
```

### M12 before index

```
{
  returned: 887,
  examined: 7144,
  ms: 47
}
```

### M13 after index

```
{
  returned: 887,
  examined: 887,
  ms: 30
}
```

### M14

```
[
  {
    v: 2,
    key: {
      _id: 1
    },
    name: '_id_'
  },
  {
    v: 2,
    key: {
      type: 1,
      ts: -1
    },
    name: 'type_1_ts_-1'
  },
  {
    v: 2,
    key: {
      productId: 1
    },
    name: 'productId_1'
  },
  {
    v: 2,
    key: {
      city: 1,
      type: 1
    },
    name: 'city_1_type_1'
  }
]
```

### report database cleaned

```

```